

Bull Boom Trading Platform - Complete Documentation & Status

1. Project Overview

Bull Boom is an AI-powered trading platform built on the MERN stack (MongoDB, Express.js, React, Node.js). The platform currently supports:

- User Authentication (Signup, Login, OTP Verification, Forgot Password)
 - User Profile Management
 - Personalized Watchlist
 - Dashboard, Orders, Positions, AI Analysis, Education, and Community Pages (UI Ready)
 - Cloudinary integration for profile images
-

2. Tech Stack

2.1 Backend

Technology	Version	Purpose
Node.js	-	Server runtime
Express.js	^4.21.2	API framework
MongoDB	4.0	Database
Mongoose	^8.9.2	ODM for MongoDB
JWT	^9.0.2	Authentication
Bcryptjs	^2.4.3	Password Hashing
Cloudinary	^1.41.3	Cloud storage for images
Multer	^2.1.1	File upload middleware
Multer-Storage-Cloudinary	^4.0.0	Cloudinary storage engine
Nodemailer	^6.9.16	Email service for OTPs
Nodemon	^3.1.9	Dev server
Morgan	^1.10.0	Logging middleware
Helmet	^8.0.0	Security headers
Cookie-Parser	^1.4.7	Cookie management

2.2 Frontend

Technology	Version	Purpose
------------	---------	---------

React	^19.2.6	UI framework
React Router DOM	^7.17.0	Routing
Tailwind CSS	^4.3.0	Styling
Framer Motion	^12.40.0	Animations
React Icons	^5.6.0	Icons
Axios	^1.17.0	HTTP client
React Hot Toast	^2.6.0	Notifications
Recharts	^3.8.1	Charts
Vite	^8.0.12	Build tool

3. Project Structure

3.1 Backend Structure

```

backend/
├── config/
│   ├── cloudinary.js      # Cloudinary config
│   ├── db.js              # MongoDB connection
│   └── multer.js          # Multer storage config
├── src/
│   ├── controllers/
│   │   ├── authController.js  # Auth endpoints logic
│   │   ├── userController.js  # User/profile endpoints
│   │   └── watchlistController.js # Watchlist endpoints
│   ├── middleware/
│   │   └── authMiddleware.js  # JWT auth middleware
│   ├── models/
│   │   ├── Otp.model.js      # OTP model
│   │   ├── User.model.js     # User model
│   │   └── Watchlist.model.js # Watchlist model
│   ├── routes/
│   │   ├── authRoutes.js     # Auth routes
│   │   ├── userRoutes.js     # User routes
│   │   └── watchlistRoutes.js # Watchlist routes
│   ├── services/
│   │   └── emailService.js    # Email sending service
│   └── utils/
│       └── generateOtp.js     # OTP generation
├── server.js                # Entry point
├── package.json
└── .env (not in git)

```

3.2 Frontend Structure

```

frontend/
├─ public/
│   └─ BullBoom.jpeg
├─ src/
│   ├─ components/
│   │   └─ Sidebar.jsx          # Sidebar component
│   ├─ layouts/
│   │   └─ DashboardLayout.jsx # Main dashboard layout
│   ├─ pages/
│   │   ├─ landing/
│   │   │   ├─ ForgotPasswordPage.jsx
│   │   │   ├─ LandingPage.jsx
│   │   │   ├─ LoginPage.jsx
│   │   │   └─ SignUpPage.jsx
│   │   ├─ AIAalysis.jsx
│   │   ├─ Community.jsx
│   │   ├─ Dashboard.jsx
│   │   ├─ Education.jsx
│   │   ├─ Orders.jsx
│   │   ├─ Positions.jsx
│   │   ├─ Profile.jsx (dynamic!)
│   │   └─ Watchlist.jsx (dynamic!)
│   ├─ routes/
│   │   └─ AppRoutes.jsx
│   ├─ services/
│   │   └─ api.js              # Axios config
│   ├─ App.jsx
│   ├─ main.jsx
│   └─ index.css
├─ package.json
└─ index.html

```

4. Data Models (MongoDB)

4.1 User Model (`User.model.js`)

Fields:

- `fullName` (required, trimmed string): User's full name
- `username` (unique, trimmed string, optional): Unique username
- `email` (required, unique, trimmed, lowercase string): User email
- `phone` (required, unique, trimmed string): User's phone number
- `password` (required, hashed string): User password
- `referralCode` (string, optional): User referral code
- `role` (enum: 'user'/'admin'): User role, default 'user'
- `isVerified` (boolean): Whether user verified email, default false
- `profileImage` (string): Cloudinary URL to profile image
- `location` (string): User location
- `tradingExperience` (enum: 'Beginner'/'Intermediate'/'Advanced'/'Professional'): User trading experience
- `riskProfile` (enum: 'Low Risk'/'Moderate Risk'/'High Risk'/'Aggressive'): User risk profile

- `membership` (enum: 'Free'/'Basic'/'Premium'/'Pro'): User membership tier

Virtual Field (Virtual)

- `profileCompletion` : Calculates profile completion percentage (based on 7 fields)

Middleware:

- Pre-save hook to hash passwords using `bcryptjs`

4.2 OTP Model (`Otp.model.js`)

Fields:

- `email` (required string): User's email
- `otp` (required string): OTP code
- `expiresAt` (Date): OTP expiration time (auto-deletes after expiry, using MongoDB TTL index)

4.3 Watchlist Model (`Watchlist.model.js`)

Fields:

- `userId` (ObjectId ref User, required): User reference
- `symbol` (string, required, uppercase): Stock/Index symbol (e.g., RELIANCE, NIFTY)
- `exchange` (enum: 'NSE'/'BSE', required, uppercase): Stock exchange
- `createdAt` , `updatedAt` : Timestamps

Index:

- `{ userId: 1, symbol: 1, exchange: 1 }`: Unique index to prevent duplicate entries per user

5. API Endpoints

5.1 Authentication Endpoints (`/api/auth`)

Method	Route	Description	Protected
POST	<code>/send-otp</code>	Send OTP to email for verification	✗
POST	<code>/verify-otp</code>	Verify OTP for signup	✗
POST	<code>/signup</code>	Sign up user	✗
POST	<code>/login</code>	Log user in and issue JWT token	✗
POST	<code>/forgot-password</code>	Send OTP for password reset	✗
POST	<code>/verify-forgot-otp</code>	Verify forgot password OTP	✗
POST	<code>/reset-password</code>	Reset user's password	✗

5.2 User Endpoints (`/api/user`)

All routes protected via JWT (`protect` middleware)

Method	Route	Description	Protected
GET	<code>/profile</code>	Get logged-in user's profile	✓
PUT	<code>/profile</code>	Update logged-in user's profile	✓
POST	<code>/upload-profile</code>	Upload profile image	✓
DELETE	<code>/account</code>	Delete user's account and Cloudinary profile image	✓

5.3 Watchlist Endpoints (`/api/watchlist`)

All routes protected via JWT (`protect` middleware)

Method	Route	Description	Protected
POST	/add	Add item to watchlist	✓
GET	/	Get logged-in user's watchlist sorted by latest first	✓
DELETE	/:id	Delete item from watchlist	✓

6. Frontend Features

6.1 Landing Page

- Hero section
- Navigation
- Links to signup/login

6.2 Authentication Pages

Signup

- Enter email
- OTP verification flow (email verification before signup)
- Password and password confirm

Login

- Email and password login
- JWT stored in localStorage and sent as Bearer token

Forgot Password

- Send OTP to email, verify OTP, reset password

6.3 Dashboard Layout

- Sidebar navigation
- Animated background
- Header with search, notifications, profile

6.4 Profile Page (Fully Dynamic!)

Features:

- Loading state while fetching
- Dynamic profile data (no hardcoded data!)
- Profile image upload (Cloudinary)
- Edit profile modal
- Profile completion progress bar and checklist
- Delete account modal
- Personal info card
- Personal info section
- Recent activity section
- All connected to MongoDB!

6.5 Watchlist Page (Fully Dynamic!)

Features:

- Summary cards
- Dynamic watchlist table
- Add symbol modal
- Search filtering
- Professional empty state
- Recently added symbols
- Watchlist stats
- All data from MongoDB!
- Prepared for future API integration (live price, change, etc.)

6.6 Other Pages (UI Ready)

- Dashboard
 - Orders
 - Positions
 - AI Analysis
 - Education
 - Community
-

7. Authentication Flow

1. **Signup:

- User enters their details
- User receives OTP to email
- OTP is verified
- User is created and JWT token issued
- JWT stored in localStorage and sent to cookies

2. Login:

- User enters credentials
- JWT is issued
- User is redirected to dashboard

3. Protected Routes:

- All `/api/user` and `/api/watchlist` use JWT
- `protect` middleware checks token

4. JWT Token:

- Stored in localStorage
 - Auto-added to all requests by ``Authorization: Bearer`
-

8. Configuration Files

8.1 Backend `server.js`

Key Features:

- Loads environment variables first!

- Connects to MongoDB
- Registers all routes: `/api/auth` , `/api/user` , `/api/watchlist`
- Enables CORS
- Uses cookie parser
- Uses Morgan for logging
- Error handlers
- Helmet security headers

8.2 Frontend `api.js`

Features:

- Axios instance with base URL
 - Request interceptor to add JWT
 - Response interceptor for error handling
-

9. Current Status

✅ Completed Features:

1. ✅ Authentication:
 - ✅ Signup/Login with JWT and OTP
 - ✅ Forgot Password flow
 - ✅ Email verification
 - ✅ JWT Authentication middleware
 2. ✅ Profile Module:
 - ✅ Full Profile UI
 - ✅ Dynamic Profile
 - ✅ Cloudinary Image Upload
 - ✅ Delete Account (with Cloudinary image removal)
 - ✅ Edit Profile
 - ✅ Profile Completion
 - ✅ All connected to MongoDB
 3. ✅ Watchlist Module:
 - ✅ Full Watchlist UI
 - ✅ Add Symbol
 - ✅ Get Watchlist
 - ✅ Delete Symbol
 - ✅ Search filtering
 - ✅ Empty state
 - ✅ All connected to MongoDB
 4. ✅ Other:
 - ✅ Landing page UI
 - ✅ Dashboard UI
 - ✅ Orders UI
 - ✅ Positions UI
 - ✅ AI Analysis UI
 - ✅ Education UI
 - ✅ Community UI
-

10. Future Tasks

1. ****Live Market Data:**
 - Integrate NSE/BSE API for real-time prices, prices and volume
 - Live market data
 - Live price change
 2. ****Watchlist enhancements**
 - Live prices on watchlist
 3. ****Orders Module**
 4. ****Positions Module**
 5. **AI Analysis Module**
 6. ****Community Module**
 7. ****Education Module**
 8. **Payment Integration**
 9. ****User Referral system**
 10. ****Admin Panel**
 11. **Testing**
 12. **Deployment**
-

11. Environment Variables (.env)

```
NODE_ENV=development/production
PORT=5000
MONGODB_URI=mongodb+srv://<username>:<password>@cluster...
JWT_SECRET=your-jwt-secret
JWT_EXPIRE=30d
EMAIL_USER=your-email@example.com
EMAIL_PASS=your-email-app-password
CLOUDINARY_CLOUD_NAME=cloud-name
CLOUDINARY_API_KEY=api-key
CLOUDINARY_API_SECRET=api-secret
CLIENT_URL=http://localhost:5173
```

12. Running the App

12.1 Backend

```
```bash
cd backend
npm install
npm run dev
```

### 12.2 Frontend

```
cd frontend
npm install
```

```
npm run dev
```

---

## 13. Documentation Authors

Created on 2026-06-11